

Documento de Arquitetura de Software: E-commerce Multi-Platform

1. Visão Geral do Projeto

O projeto consiste em uma plataforma de e-commerce completa, projetada para demonstrar proficiência em engenharia de software através da implementação de regras de negócio consistentes em múltiplos ecossistemas. A arquitetura contempla duas APIs backend intercambiáveis e três clientes frontend distintos (Web e Mobile), além de um painel administrativo.

Objetivo Principal: Comprovar domínio técnico em desenvolvimento Full-Stack, Mobile e Cloud, garantindo que diferentes tecnologias consigam consumir e expor o mesmo contrato de dados perfeitamente.

2. Stack Tecnológico

- **Backends (APIs):** Python (FastAPI) e Node.js/TypeScript (NestJS).
- **Frontend Administrativo:** React via Next.js.
- **Frontends Vitrine (Clientes):** Web (Next.js), Mobile Android/iOS (React Native e Flutter).
- **Banco de Dados:** PostgreSQL (Relacional).
- **Armazenamento de Arquivos:** Object Storage (Firebase Storage ou AWS S3).
- **Infraestrutura e DevOps:** Docker, Docker Compose e GitHub Actions (CI/CD).

3. Padrões Arquiteturais e Comunicação

- **API-First Design:** Os contratos de API (rotas, payloads, respostas e status codes) serão definidos primeiro e documentados via Swagger/OpenAPI. Ambos os backends devem respeitar rigorosamente este contrato.
- **Intercambialidade de API:** Todos os frontends possuirão um mecanismo (como uma variável de ambiente `API_PROVIDER`) para alternar facilmente as requisições entre o backend FastAPI e o NestJS.
- **Comunicação:** RESTful sobre HTTP/HTTPS, com respostas padronizadas em JSON.

4. Escopo das Aplicações

4.1. Camada de Backend (NestJS & FastAPI)

Ambas as aplicações atuarão como provedoras de dados unificadas.

- **Gerenciamento de Catálogo:** Endpoints para CRUD de Produtos, Categorias e Subcategorias.
- **Gerenciamento de SKUs (Variações):** Lógica central para associar Tamanhos, Cores e Imagens específicas a um mesmo Produto (SKU).
- **Orquestração de Mídia:** Geração de URLs assinadas ou integração direta para upload de imagens no Storage em nuvem.

4.2. Camada Frontend: Painel Administrativo (Next.js)

Aplicação de uso interno, protegida por autenticação.

- **Gestão de Estoque:** Interfaces para cadastro complexo de produtos com múltiplas variações (SKUs).
- **Upload de Mídia:** Interface para envio das fotos dos produtos e vinculação das mesmas às cores/variantes correspondentes.

4.3. Camada Frontend: Vitrines (Next.js, React Native, Flutter)

Aplicações focadas no consumidor final, priorizando usabilidade e performance.

- **Listagem e Filtros:** Exibição do catálogo navegando por categorias e subcategorias.
- **Página de Produto Dinâmica (Core Feature):** O estado da aplicação (gerenciado via Zustand, Provider, BLoC, etc.) deve reagir à seleção do usuário. Ao clicar em uma cor ou tamanho diferente, a interface deve buscar o SKU correspondente e atualizar a galeria de imagens e o preço dinamicamente, sem recarregar a tela.

5. Requisitos Não Funcionais

5.1. Segurança (Security)

- **Autenticação:** Implementação de JWT (JSON Web Tokens) com controle de acesso baseado em roles (RBAC) para proteger as rotas administrativas.
- **Proteção de Dados:** Sanitização de inputs via `class-validator`/`Pydantic` para prevenir injeção de SQL e XSS.
- **Rede:** Configuração estrita de CORS e Rate Limiting nas APIs.

5.2. Qualidade e Testes (QA)

- **Backend:** Cobertura de regras de negócio críticas via Testes Unitários (Jest/PyTest) e Testes de Integração nas rotas da API com banco em memória.
- **Frontend/Mobile:** Testes de Componente para validar a lógica de troca de estado (ex: selecionar cor - > mudar foto) e pelo menos um teste E2E (End-to-End) simulando o fluxo de compra/visualização do usuário.

5.3. Infraestrutura e Deploy (DevOps)

- **Isolamento:** Uso de containers Docker para cada serviço (FastAPI, NestJS, Postgres, Admin).
- **Orquestração Local:** `docker-compose.yml` para levantar todo o ecossistema com um único comando.
- **Automação:** Pipelines de CI no GitHub Actions para execução automatizada de linters e suítes de testes a cada push ou Pull Request.